

교과목 3 : 초거대언어모델(LLM)

단원 3 : LLM

DAY1. LLM의 개념과 API

목 차

1. LLM 이해		3
2. LLM의 사용		7
3. API 방식 LLM		10
4. ChatGPT 활용		15
5. FastAPI 사용법		25

1. LLM 이해

GPT-3 와 초거대언어모델의 등장

LLM 은 **Large Language Model**, 즉 **대규모 언어 모델**입니다.

GPT-3 는 2020 년에 공개된 대표적인 초거대언어모델이며, **1,750 억 개 파라미터**를 가진 자기회귀 언어 모델입니다. GPT-3 의 중요한 의미는 모델 크기를 크게 키우면 별도의 세부 학습 없이도 질문 답변, 번역, 요약, 글쓰기 같은 다양한 작업을 수행할 수 있다는 점을 보여준 것입니다. ([OpenAI](#))

기존 NLP 모델은 보통 작업별로 따로 학습해야 했습니다.

- 감성 분석 모델
- 번역 모델
- 요약 모델
- 질문 답변 모델

하지만 GPT-3 이후의 LLM 은 하나의 모델이 프롬프트만 바뀌도 여러 작업을 수행할 수 있게 되었습니다.

LLM 에서의 Pre-training 단계

Pre-training 은 LLM 을 만들 때 가장 먼저 수행하는 대규모 사전 학습 단계입니다.

모델은 인터넷 문서, 책, 코드, 위키 문서 등 대량의 텍스트를 학습하면서 다음 토큰을 예측합니다.

- 입력: 인공지능은 데이터를 학습하여
- 정답: 패턴

이 과정을 매우 큰 데이터로 반복하면 모델은 다음 능력을 얻게 됩니다.

- 문장 구조 이해
- 단어 의미 관계 학습
- 문맥 파악
- 상식적 표현 학습
- 코드 패턴 학습
- 질문과 답변 구조 학습

즉, Pre-training 은 LLM 이 언어의 기본 능력을 갖추는 과정입니다.

LLM 의 종류와 차이

LLM 은 구조와 목적에 따라 나눌 수 있습니다.

종류	설명	예시
Decoder-only 모델	이전 토큰을 보고 다음 토큰을 생성	GPT 계열
Encoder-only 모델	문장을 이해하고 분류하는 데 강함	BERT 계열
Encoder-Decoder 모델	입력 문장을 다른 문장으로 변환	T5, BART
Instruction-tuned 모델	사람의 지시를 잘 따르도록 추가 학습	InstructGPT, ChatGPT 계열
Multimodal 모델	텍스트뿐 아니라 이미지, 음성 등도 처리	GPT-4o 계열

GPT 계열은 주로 **생성형 작업**에 강합니다.

- 글쓰기
- 답변 생성
- 코드 작성
- 요약
- 번역
- 챗봇

BERT 계열은 주로 **이해 기반 작업**에 강합니다.

- 문장 분류
- 감성 분석
- 개체명 인식
- 문장 유사도
- 검색 랭킹

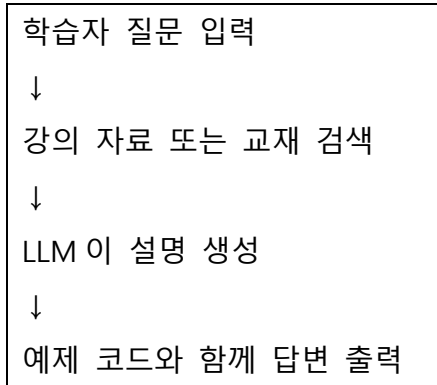
LLM 의 다양한 활용 사례

LLM 은 실무에서 다음과 같이 사용됩니다.

- 고객 상담 챗봇
- 문서 요약 시스템
- 강의 자료 자동 생성
- 코드 자동 작성
- SQL 자동 생성
- 보고서 작성 보조

- 이메일 초안 작성
- PDF 질의응답
- 사내 지식 검색
- 번역 서비스
- 데이터 분석 보조

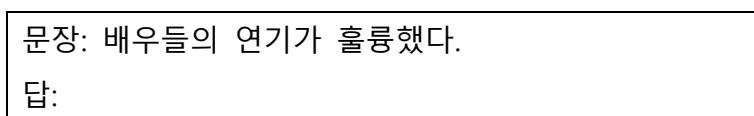
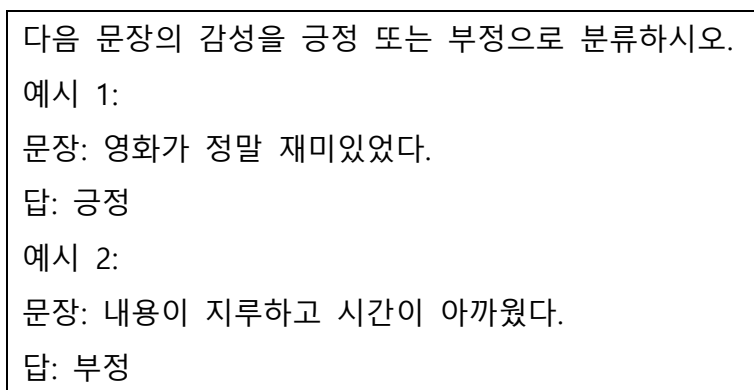
예를 들어 교육 분야에서는 다음과 같이 사용할 수 있습니다.



인컨텍스트 러닝

In-context Learning 은 모델을 다시 학습시키지 않고, 프롬프트 안에 설명이나 예시를 넣어 원하는 작업을 수행하게 하는 방식입니다.

예를 들어 감성 분석을 시킨다고 할 때:



모델은 앞의 예시를 보고 마지막 문장의 답을 "긍정"으로 생성합니다.

이 방식의 장점은 별도 학습 데이터셋을 만들거나 모델을 재학습하지 않아도 된다는 점입니다.

LLM 의 환각 현상과 보완 방식

환각 현상 Hallucination 은 LLM 이 사실이 아닌 내용을 그럴듯하게 생성하는 현상입니다.

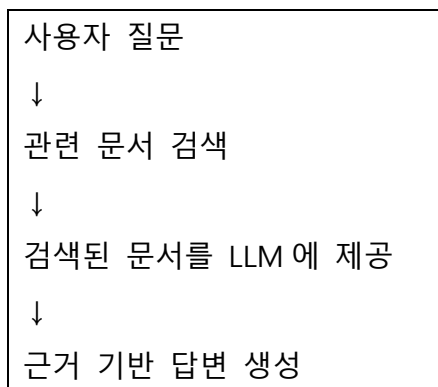
예:

- 존재하지 않는 논문 제목 생성
- 틀린 법률 조항 설명
- 없는 API 함수 사용
- 잘못된 코드 작성
- 출처 없는 통계 제시

보완 방법은 다음과 같습니다.

- RAG 적용
- 공식 문서 검색 후 답변
- DB 조회 결과 기반 답변
- 출처 표시
- Function Calling 사용
- 검증 로직 추가
- 응답 후처리
- 사람 검토 단계 추가

특히 실무에서는 LLM 에게 모든 것을 맡기기보다, 외부 데이터와 연결하는 구조가 중요합니다.



이 방식이 RAG 입니다.

2. LLM의 사용

프롬프트의 개념

프롬프트 Prompt 는 LLM 에게 전달하는 입력 지시문입니다.

프롬프트는 단순 질문일 수도 있고, 역할·조건·출력 형식을 포함한 상세 명령일 수도 있습니다.

나쁜 프롬프트:

LLM 설명해줘.

좋은 프롬프트:

너는 인공지능 강사이다.

LLM 의 개념을 자연어 처리 수업을 듣는 학습자가 이해할 수 있도록 설명하시오.

다음 항목을 포함하시오.

1. LLM 의 정의
2. 학습 방식
3. Transformer 구조
4. API 활용 예시
5. 실무 활용 사례

프롬프트 품질이 좋아지면 답변 품질도 좋아집니다.

Zero-shot, One-shot, Few-shot

LLM 사용 방식은 예시 개수에 따라 구분할 수 있습니다.

Zero-shot

예시 없이 바로 지시합니다.

다음 문장의 감성을 긍정 또는 부정으로 분류하시오.

문장: 이 제품은 배송도 빠르고 품질도 좋다.

답:

One-shot

예시를 1 개 제공합니다.

문장: 영화가 재미있었다.

답: 긍정

문장: 음식이 너무 짭다.

답:

Few-shot

예시를 여러 개 제공합니다.

문장: 정말 만족스러운 서비스였다.

답: 긍정

문장: 다시는 구매하지 않을 것이다.

답: 부정

문장: 가격 대비 성능이 좋다.

답: 긍정

문장: 제품이 고장 난 상태로 도착했다.

답:

Few-shot 은 모델에게 원하는 답변 패턴을 보여줄 수 있어 결과가 더 안정적입니다. GPT-3 논문에서도 모델을 큰 규모로 확장하면 few-shot 성능이 크게 향상된다는 점을 강조했습니다. ([OpenAI](#))

Playground 를 활용한 실습

Playground 는 코드를 작성하지 않고 웹 화면에서 LLM API 동작을 테스트할 수 있는 실습 도구입니다.

Playground 에서 할 수 있는 작업은 다음과 같습니다.

1. 모델 선택
2. 프롬프트 입력
3. 응답 확인
4. temperature 조절
5. 출력 길이 조절

6. system/user 메시지 테스트

7. API 호출 형식 확인

주의할 점은 Playground 도 실제 API 호출과 같은 방식으로 동작하므로, 사용한 토큰은 API 사용량과 과금에 포함될 수 있다는 점입니다. ([OpenAI Help Center](#))

실습 순서 예시는 다음과 같습니다.

1. Playground 접속
2. 모델 선택
3. System 메시지 작성
4. User 메시지 입력
5. 실행
6. 응답 비교
7. temperature 변경
8. zero-shot, one-shot, few-shot 결과 비교

3. API 방식 LLM

API 방식 LLM 호출

API 방식은 Python, JavaScript, Java, Spring Boot, Streamlit 같은 프로그램에서 LLM 을 호출하는 방식입니다.

기본 흐름은 다음과 같습니다.

```
사용자 입력
↓
서버 또는 앱에서 API 요청 생성
↓
LLM 서버가 응답 생성
↓
응답을 앱 화면에 출력
```

Python 예시는 다음과 같습니다.

```
import os
from openai import OpenAI

# 환경변수에서 API Key 를 읽어 OpenAI 클라이언트를 생성합니다.
# API Key 를 코드에 직접 작성하면 GitHub 에 노출될 수 있으므로 환경변수를 사용하는 것이
안전합니다.
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

# LLM 에게 전달할 질문을 문자열로 작성합니다.
user_input = "LLM 에서 zero-shot 과 few-shot 의 차이를 설명해줘."

# Responses API 를 사용하여 모델에게 응답 생성을 요청합니다.
response = client.responses.create(
    model="gpt-5.5",
    input=user_input
)
```

```
# 모델이 생성한 최종 텍스트 응답을 출력합니다.
print(response.output_text)

Streamlit 앱에서는 다음처럼 연결할 수 있습니다.
import os
import streamlit as st
from openai import OpenAI

# OpenAI API 클라이언트를 생성합니다.
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

# 웹 화면 제목을 표시합니다.
st.title("LLM API 실습 챗봇")

# 사용자가 질문을 입력할 수 있는 입력창을 만듭니다.
question = st.text_input("질문을 입력하세요")

# 질문이 입력되었을 때만 API 를 호출합니다.
if question:
    # 사용자의 질문을 LLM API 에 전달합니다.
    response = client.responses.create(
        model="gpt-5.5",
        input=question
    )

    # LLM 의 응답을 Streamlit 화면에 출력합니다.
    st.write(response.output_text)
```

API 호출의 과금 방식

LLM API 과금은 보통 **토큰 사용량**을 기준으로 계산됩니다.

토큰은 크게 다음으로 나뉩니다.

- Input 토큰: 사용자가 API 에 보낸 입력
- Output 토큰: 모델이 생성한 응답
- Cached 토큰: 이전 입력이 캐시되어 할인 적용될 수 있는 토큰

OpenAI API 사용량은 모델별, 입력/출력/캐시 토큰 유형별로 가격이 다르게 적용됩니다.

([OpenAI Help Center](#))

예를 들어 다음과 같은 요청이 있다고 가정합니다.

- 입력 프롬프트: 1,000 토큰
- 모델 응답: 500 토큰

총 사용량은 다음과 같습니다.

- 총 사용 토큰 = Input 토큰 1,000 + Output 토큰 500
- 총 사용 토큰 = 1,500 토큰

비용 절감 방법은 다음과 같습니다.

- 질문과 관련 없는 긴 문서 제거
- 출력 길이 제한
- 요약본 사용
- RAG 로 필요한 문단만 전달
- 반복되는 시스템 프롬프트 최소화
- 저렴한 모델과 고성능 모델을 구분 사용
- Batch API 사용 검토

Input 토큰과 Output 토큰

Input 토큰은 모델에게 전달하는 모든 입력입니다.

포함되는 예:

- System 메시지
- User 메시지
- 예시 문장
- 검색된 문서
- 이전 대화 기록
- 코드
- 표 데이터

Output 토큰은 모델이 생성하는 응답입니다.

포함되는 예:

- 설명 문장
- 코드
- 요약 결과
- 번역 결과
- 분석 결과
- JSON 응답

예를 들어:

입력:

"다음 문장을 영어로 번역하시오: 나는 오늘 학교에 간다."

출력:

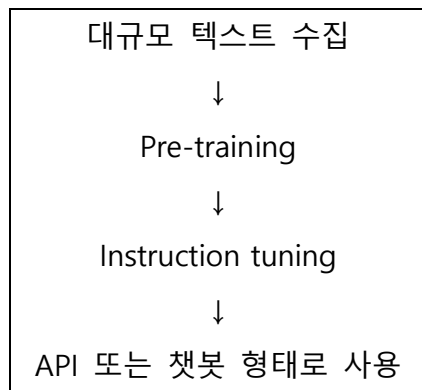
"I am going to school today."

입력 문장은 Input 토큰으로 계산되고, 번역 결과는 Output 토큰으로 계산됩니다.

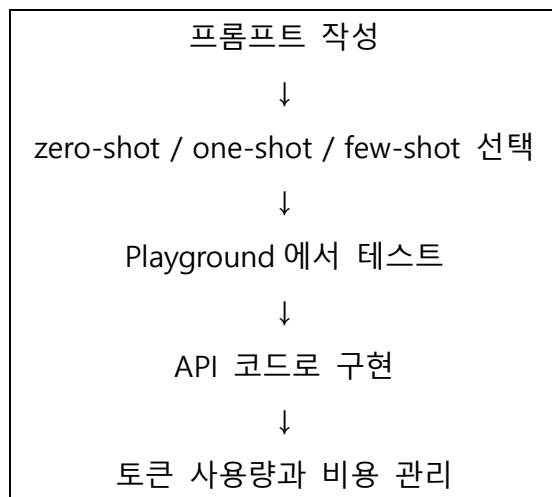
토큰은 단어와 완전히 같지 않습니다. 영어는 짧은 단어 단위로 나뉘는 경우가 많고, 한국어는 조사나 어절, 부분 문자열 단위로 나뉠 수 있습니다.

OpenAI 도움말에서는 토큰이 단어 일부나 전체에 해당할 수 있으며, API 비용은 입력과 출력 토큰을 기준으로 계산된다고 설명합니다. ([OpenAI Help Center](#))

LLM 학습 흐름:



LLM 사용 흐름:



LLM 을 실무에 적용할 때 가장 중요한 것은 다음 세 가지입니다.

- 좋은 프롬프트 작성
- 정확한 외부 데이터 연결
- 토큰과 비용 관리

4. ChatGPT 활용

← ChatGPT 접속하기

❖ ChatGPT를 만든 회사는 OpenAI 입니다.

- 공식 웹사이트: openai.com
- "기술 설명, 최신 모델 발표, API 문서, 기업 안내 제공 "

❖ 우리가 사용하는 ChatGPT 대화 서비스는

- 기존 주소: chat.openai.com
- 현재 주소: chatgpt.com (독립 브랜드로 변경됨)
- chat.openai.com 으로 접속해도 자동으로 chatgpt.com 으로 이동

❖ 무료 vs 유료, 무엇이 다를까?

항목	무료 버전 (기본: GPT-5 제한 모드)	유료 버전 (Plus, \$20/월)
사용 모델	GPT-5 (Instant, Thinking mini 중심 제공)	GPT-5 전체 모드 (Auto, Instant, Thinking mini, Thinking) + GPT-4 계열 선택 가능
모델 선택	❌ 불가 (제한된 GPT-5 모드만 사용)	✅ 가능 (GPT-5 전체 모드, GPT-4o, GPT-4.1, o3, o4-mini 등)
GPT-5 내부 모드	- Instant (즉시 응답)- Thinking mini (짧은 사고, 빠른 응답)	- Auto (상황별 자동)- Instant- Thinking mini- Thinking (깊이 있는 응답)
이미지 생성	⚠️ 일부 가능 (기본 품질, 제한 있음)	✅ 고해상도, 스타일/옵션 선택 가능
파일 업로드	⚠️ 일부 가능 (소형 PDF/텍스트 위주)	✅ 분석/질문/표 변환 등 고급 활용 가능
음성 대화	⚠️ 기본 음성만 (1~2개)	✅ 고급 음성 5종, 실시간 대화
맞춤형 GPT 사용	✅ 사용 가능 (기본 템플릿 한정)	✅ GPT 생성/배포 등 전체 기능 활용
웹 검색	✅ 가능 (제한 있음)	✅ 더 빠르고 정확한 검색 환경 제공

요금제 이해

- Plus는 모든 일반 사용자에게 충분한 고급 기능 제공
- Pro는 AI 연구, 복잡한 자동화 작업, 기업용 개발자를 위한 하이엔드 버전

The screenshot shows the ChatGPT pricing page with the title '플랜 업그레이드' (Plan Upgrade). It features three main plan cards: Free, Plus, and Pro. The 'Plus' plan is highlighted with a purple background and a '만족' (Satisfied) badge. Below the plans, there is a section for '비즈니스에 더 많은 기능이 필요하신가요?' (Need more features for business?) with a link to 'ChatGPT Enterprise 보기' (View ChatGPT Enterprise).

Free	Plus	Pro
\$0 USD / 월	\$20 USD / 월	\$200 USD / 월
일상적인 작업에 사용하는 만능리전스	고급 만능리전스에 더 많은 액세스	최고의 ChatGPT에 완전한 액세스
나의 현재 계층	Plus 이용하기	Pro 이용하기
- GPT-5에 액세스 - 제한적인 파일 업로드 - 느린 속도에 제한적 이미지 생성 - 제한적 메모리 및 컨텍스트 - 제한적 상용 리서치	- 고급 이상을 사용하는 GPT-5 - 메시지 및 업로드 확장 - 보다 빠른 속도, 더 확장된 이미지 생성 - 메모리 및 컨텍스트 확장 - 상용 리서치 및 에이전트 모드 확장 - 프로젝트, 작업, 맞춤형 GPT - Sora 영상 생성 - Codex 에이전트	- Pro 이상을 사용하는 GPT-5 - 제한 없는 메시지 및 업로드 - 보다 빠른 속도에 제한 없는 이미지 생성 - 메모리 및 컨텍스트 최대 한도 - 상용 리서치 및 에이전트 모드 최대 한도 - 프로젝트, 작업, 맞춤형 GPT 확장 - Sora 영상 생성 확장 - Codex 에이전트 확장 - 새 기능 리서치 프리뷰

기본 플랜에 없는 기능: 음성, 도구들, 보기

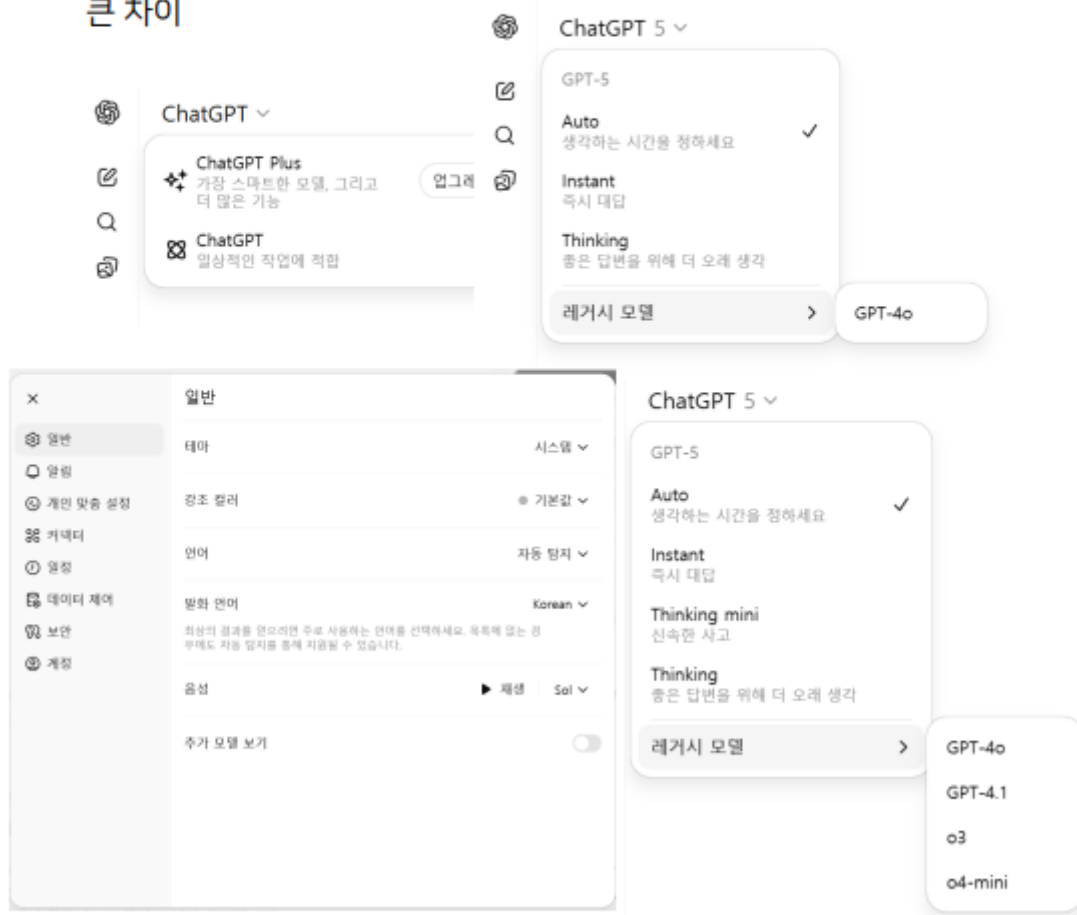
무제한 액세스의 경우에도 요율을 방지 조치의 적용 대상이 될 수 있습니다. 자세한 내용은 보기

비즈니스에 더 많은 기능이 필요하신가요?
ChatGPT Enterprise 보기

대한민국

← GPT 무료/유료 플랜별 모델 변경

- 무료는 GPT-5의 일부 모드 체험에 초점이 맞춰져 있음
- 유료는 GPT-5 전체 모드와 GPT-4 계열까지 자유롭게 선택 가능
- '추가 모델 보기'를 통해 과거 모델도 필요에 따라 활용할 수 있다는 점이 큰 차이



← ChatGPT 모델 살펴보기

모델명	출시일	무료 사용자 제공	유료 사용자 제공	주요 특징
GPT-3.5	2022-11	×	×	초기 ChatGPT 모델, 현재는 API 전용 사용 가능
GPT-4	2023-03	×	×	지원 종료 (초기 GPT-4 성능 모델)
GPT-4 Turbo	2023-11	×	×	지원 종료 (GPT-4o로 대체됨)
GPT-4o	2024-05	×	×	최초의 멀티모달(ChatGPT 음성·이미지) 모델, 현재는 제공 안 됨
o3-mini	2025-01	×	×	Reasoning 특화 경량 모델 , 빠른 추론·계산 작업용
o3	2025-04	×	✓	고급 Reasoning 모델, 복잡한 추론·문제 해결에 강점
o4-mini	2025-04	×	✓	최신 경량화 모델, 빠른 속도·낮은 비용, 일상적 대화/서비스용
GPT-4.1	2025-01	×	✓	GPT-4 개선판, 안정성과 정밀성 강화
GPT-4.1 mini	2025-04	×	✓	GPT-4.1 기반 경량 모델, 속도·비용 효율 최적화
GPT-5	2025-05	✓ (제한 모드)	✓ (전체 모드)	최신 모델, Auto / Instant / Thinking mini / Thinking 모드 제공

← ChatGPT 무료 플랜 제약사항 이해하기

❖ 모델 및 성능 제한

- 모델 고정: 무료 사용자는 GPT-5 제한 모드(Instant / Thinking mini)만 제공
- 선택 불가: GPT-4.1, o3, o4-mini, GPT-5 Thinking 모드 전체 등은 Plus 이상에서만 가능

❖ 이미지 생성 제한

- 대기 메시지: "많은 사람들이 현재 이미지를 생성하고 있어 시간이 걸릴 수 있습니다."
- 품질 제한: 기본 해상도만 제공, 고해상도·스타일 옵션은 유료 전용
- 생성 횟수: 요청 횟수가 제한될 수 있음

❖ 파일 업로드 제한

- 용량 제약: 소형 PDF / 텍스트 파일만 지원
- 복잡한 분석 불가: 표 변환, 긴 문서 요약 등은 Plus 전용

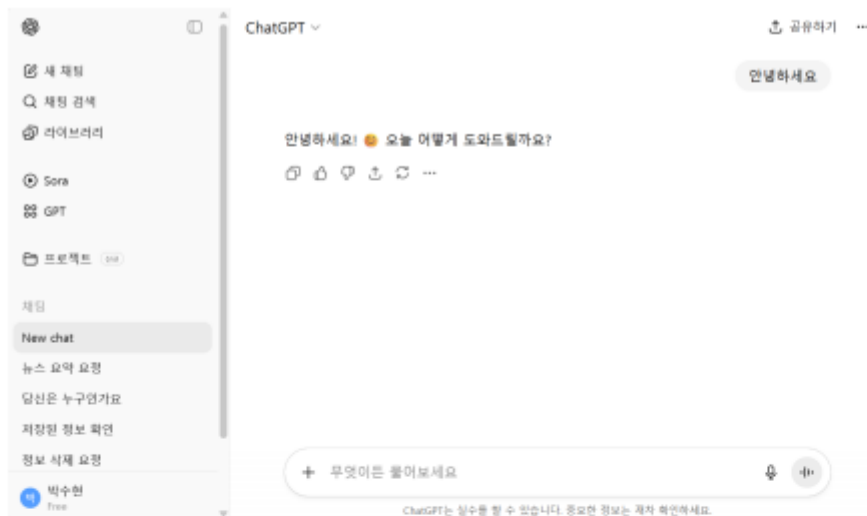
❖ 응답 속도 및 사용량

- 속도 제한: 답변이 더 느리게 도착할 수 있음
- 사용량 제한: 일정 횟수 이상 사용 시 "잠시 후 다시 시도하세요" 메시지 발생

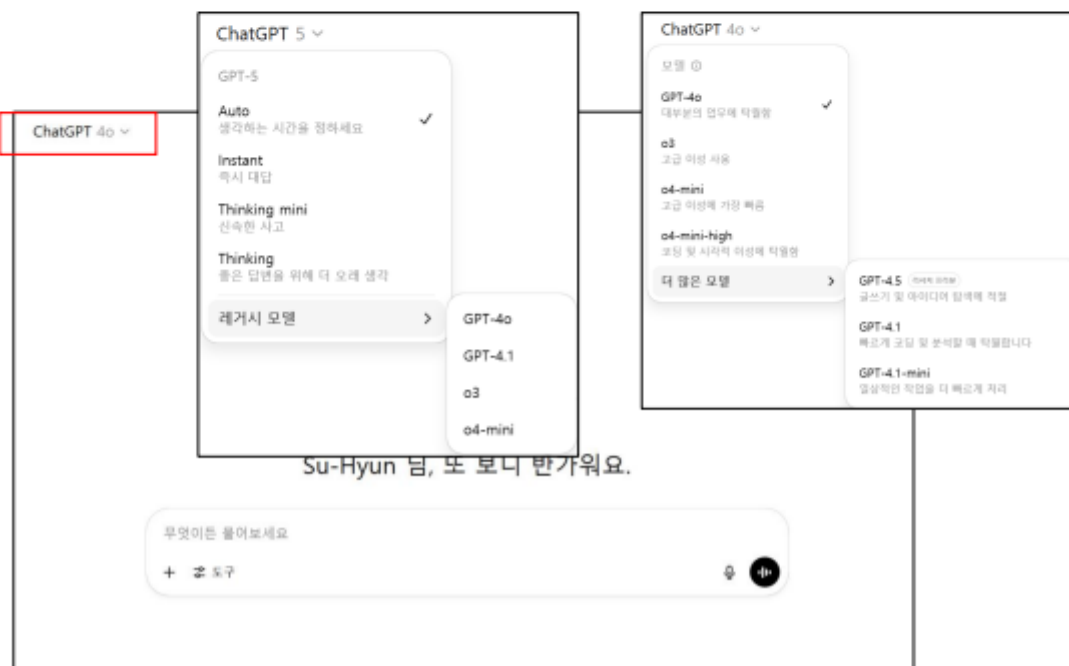
❖ 웹 검색 제한

- 무료 사용자도 웹 검색 가능하나, 속도·정확도에 제약
- 일부 최신 정보 반영에 지연 발생

← 화면 구성 한눈에 보기



← 유료 사용자의 추가적인 모델 선택 기능



← ChatGPT는 내 정보를 기억할까?

ChatGPT ▾

🔗 공유하기

나는 누구야?

제가 직접적으로 당신이 누구인지 알 수는 없어요. 😊
다만 이전 대화에서 알려주신 정보를 기반으로 말씀드리면, **당신의 이름은 "박수현"이
라고 하셨던 기억이 있어요.

원하시면 제가 지금까지 저장된 정보를 정리해서 알려드릴까요?

📄 📌 🔄 🗑️ ⋮

×

🏠 일반

🗨️ 알림

🔒 개인 맞춤 설정

⚙️ 연결된 앱

🔒 데이터 제어

🔒 보안

👤 계정

개인 맞춤 설정

맞춤형 지침 커짐 >

메모리 ⓘ

저장된 메모리 참고

ChatGPT가 응답할 때 메모리를 저장하고 사용하도록 합니다.

☒

채팅 기록 참고

ChatGPT가 응답할 때 최근 대화를 참고하도록 합니다.

☒

메모리 관리하기

관리

ChatGPT가 Bing과 같은 검색 제공자 사용 시 메모리를 사용해 질문을 개인 맞춤 설정합니다. [자세히 알아보기](#)

저장된 메모리

×

ChatGPT는 최근 채팅의 대부분을 기억하여 빠르면 시간이 지나면 잊는 것이 정상입니다. 저장된 메모리는 질문 몇 개씩입니다. [프라이빗 모드](#)

ChatGPT가(가) 채팅할 때, 저장하는 세부 사항과 기본 설정이 여기에 표시됩니다.

모두 삭제

ChatGPT에도 시크릿 모드가 있다? - 임시 채팅 기능



임시 채팅

- 기록에 남지 않음**
임시 채팅은 귀하의 기록에 나타나지 않습니다. 안전을 위해, 우리는 최대 30일 동안 귀하의 채팅 복사본을 보관할 수 있습니다.
- 모델 훈련에 사용되지 않음**
임시 채팅은 우리의 모델을 개선하는 데 사용되지 않습니다.
- 메모리 꺼짐**
임시 채팅 내에서는 ChatGPT가 메모리를 사용하거나 업데이트하지 않습니다. 맞춤형 지점을 사용하는 경우에는 그대로 따르게 합니다.

[계속](#)



← ChatGPT 실습을 위한 환경 준비하기

❖ 줄바꿈: Shift + Enter

- 단순 Enter는 바로 전송되므로, 프롬프트가 길거나 여러 문단일 경우에는 반드시 줄바꿈 키를 활용해야 합니다.

무엇을 도와드릴까요?

1. 다음 내용을 요약해줘.
2. 핵심 단어 5개도 추출해줘.
3. 마지막으로 한 문장 의견도 덧붙여줘.

+ ⌂ 도구

🗨

↑

❖ 마크다운 표현

- ChatGPT는 기본적으로 마크다운을 지원합니다.
- 예: 굵게, - 리스트, 1. 번호 등
- 구조적 응답을 원할 경우 마크다운을 활용하면 정리된 답변을 유도할 수 있습니다.

← 대화 흐름, 어떻게 관리할까?

❖ 새 대화(New Chat)의 역할

- 이전 대화의 맥락이 이어지는 구조
- 실험이나 새로운 주제는 새 대화로 시작

❖ 왜 중요할까?

- GPT는 이전 대화 맥락을 기억합니다.
- → 과거 질문이 현재 응답에 영향을 줄 수 있음
- 실험의 정확도를 위해서는 항상 새 대화로 시작하는 습관 필요

❖ 기존 대화 관리 팁

- 제목은 첫 질문 기준 자동 생성
- 제목 클릭 → 이전 대화 열람
- 점 세 개(...) 클릭 → 이름 바꾸기 / 삭제 / 고정 가능

개인맞춤 설정

❖ 답변 길이 조정

- ✓ “가능하면 간결하고 핵심만 요약해 주세요” 같은 맞춤 설정을 넣으면, 프롬프트 변화를 실험할 때 결과 비교가 훨씬 쉬워집니다.

❖ 답변 스타일 지정

- ✓ “리스트 중심으로 정리해 주세요”
- ✓ “항상 예시는 1개만 들어주세요”
→ 이런 식으로 설정해 두면 답변이 지나치게 장황해지는 걸 방지할 수 있습니다.

❖ 역할과 관점 지정

- ✓ “프롬프트 분석을 도와주는 조교처럼 간단명료하게 설명해 주세요”
- ✓ 특정 직업이나 상황을 가정하면, 실험에서 의도된 응답 스타일을 일정하게 유지할 수 있습니다.

5. FastAPI 사용법

1. FastAPI 란?

FastAPI 는 Python 으로 REST API 와 웹 백엔드 서버를 매우 쉽고 빠르게 개발할 수 있도록 만들어진 고성능 웹 프레임워크입니다.

FastAPI 는 Python 의 타입 힌트(Type Hint) 기능을 적극 활용하여 코드의 가독성을 높이고, 자동 데이터 검증과 API 문서 생성을 제공합니다. 또한 비동기(Asynchronous) 프로그래밍을 지원하여 많은 사용자의 요청을 효율적으로 처리할 수 있습니다.

FastAPI 는 2018 년에 Sebastián Ramírez 가 개발하였으며, 현대적인 Python 웹 프레임워크로 널리 사용되고 있습니다.

2. FastAPI 의 특징

FastAPI 의 대표적인 특징은 다음과 같습니다.

특징	설명
높은 성능	비동기(async/await)를 지원하여 매우 빠른 처리 속도를 제공
쉬운 개발	코드가 간결하고 이해하기 쉽다.
자동 문서 생성	Swagger UI 와 ReDoc 을 자동 생성
데이터 검증	Pydantic 을 이용하여 입력 데이터를 자동 검증
타입 힌트 지원	Python Type Hint 를 그대로 사용
REST API 개발	RESTful API 구현에 매우 적합
비동기 프로그래밍	async/await 완전 지원
ORM 연동	SQLAlchemy 등 다양한 ORM 과 연동 가능
JWT 인증	로그인 인증 기능 구현 가능
OpenAPI 지원	OpenAPI 표준 문서 자동 생성

3. FastAPI 가 등장한 이유

기존 Python 웹 프레임워크에는 다음과 같은 것들이 많이 사용되었습니다.

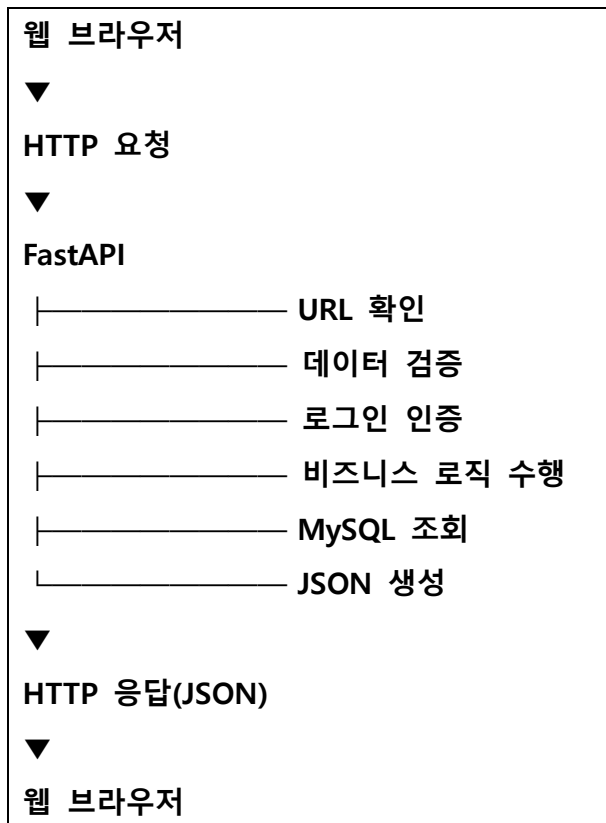
프레임워크	특징
Flask	가볍지만 많은 기능을 직접 구현해야 함
Django	기능이 매우 많지만 다소 무거움
FastAPI	빠르고 자동화 기능이 많음

FastAPI 는

- Flask 보다 생산성이 높고
- Django 보다 가볍고
- 비동기 처리가 가능하며
- Swagger 문서를 자동 생성한다는 장점 때문에 많은 프로젝트에서 사용됩니다.

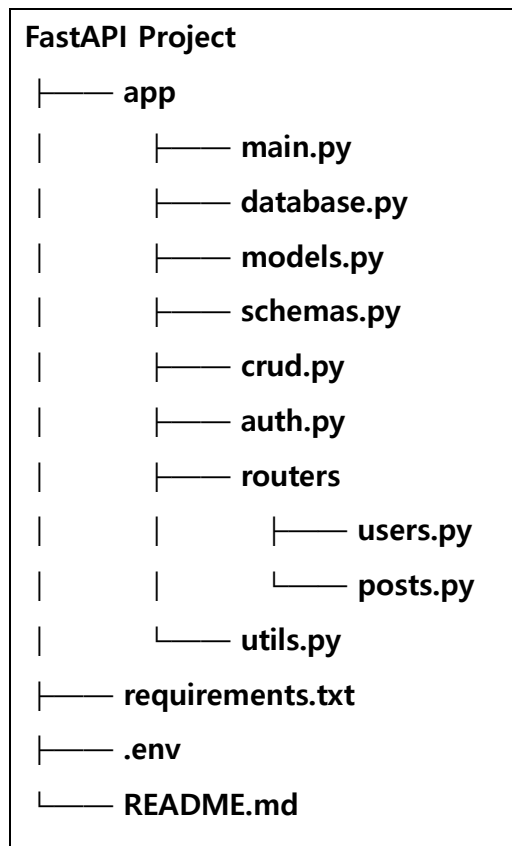
4. FastAPI 의 동작 구조

FastAPI 의 전체 동작 과정은 다음과 같습니다.



5. FastAPI 의 구성 요소

FastAPI 프로젝트는 일반적으로 다음과 같이 구성됩니다.



각 파일의 역할은 다음과 같습니다.

파일	역할
main.py	FastAPI 시작 파일
database.py	MySQL 연결
models.py	데이터베이스 테이블 모델
schemas.py	요청/응답 데이터 정의
crud.py	데이터베이스 처리
auth.py	로그인 인증
routers	API 주소 관리

6. FastAPI 의 주요 기능

FastAPI 는 다음과 같은 기능을 제공합니다.

① REST API 개발

예를 들어

GET /posts : 게시글 목록 조회

GET /posts/1 : 게시글 상세조회

POST /posts : 게시글 등록

PUT /posts/1 : 게시글 수정

DELETE /posts/1 : 게시글 삭제

② 자동 데이터 검증

회원가입 요청

```
{  
    "username": "hong",  
    "password": "1234"  
}
```

FastAPI 는 입력값을 자동 검사합니다.

예를 들어

username: str

age: int

이라고 작성하면

```
{  
    "username": "hong",  
    "age": "abc"  
}
```

와 같이 잘못된 자료형이 들어오면 자동으로 오류를 반환합니다.

③ JSON 자동 변환

Python 객체

```
{  
    "name": "홍길동",  
    "age": 25  
}
```

자동으로

```
{  
    "name": "홍길동",  
    "age": 25  
}
```

형태의 JSON 응답으로 변환됩니다.

④ 비동기 처리

FastAPI 는 `async`, `await` 를 이용하여 동시에 많은 요청을 처리할 수 있습니다.

예를 들어

100 명의 사용자가 동시에 접속

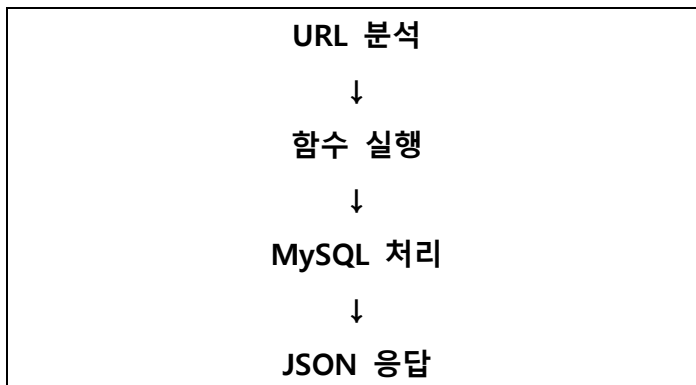
↓

비동기로 처리하여 응답 속도가 향상됩니다.

7. FastAPI 의 실행 과정

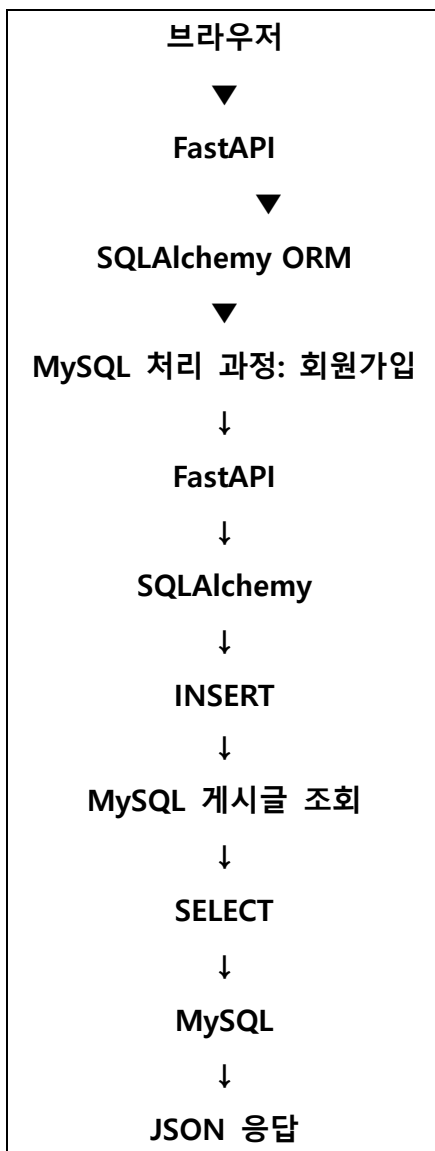
프로그램을 실행하면 `main.py` 부터 시작합니다.

```
↓  
app = FastAPI()  
↓  
API 등록  
↓  
웹 서버 실행  
↓  
클라이언트 요청  
↓
```



8. FastAPI 와 MySQL 연동 구조

실무에서는 보통 다음과 같이 연결합니다.



9. Swagger 자동 생성

FastAPI 의 가장 큰 장점 중 하나는 Swagger UI 를 자동으로 생성한다는 것입니다.

서버 실행 후 <http://127.0.0.1:8000/docs> 로 접속하면

- API 목록
- 요청(Request)
- 응답(Response)
- 테스트 기능

을 웹 화면에서 바로 확인할 수 있습니다.

또한 <http://127.0.0.1:8000/redoc> 에서는 ReDoc 형식의 API 문서도 자동으로 제공합니다.

10. FastAPI 의 장점

- Python 문법이 간결하여 개발 속도가 빠릅니다.
- 비동기 처리로 높은 성능을 제공합니다.
- Swagger UI 와 ReDoc 을 자동 생성합니다.
- 입력 데이터를 자동 검증하여 오류를 줄일 수 있습니다.
- SQLAlchemy 등 ORM 과 쉽게 연동됩니다.
- JWT 기반 로그인과 같은 인증 기능을 구현하기 쉽습니다.
- OpenAPI 표준을 기본 지원하여 다른 시스템과의 연동이 용이합니다.

11. FastAPI 의 단점

- Django 처럼 관리자(Admin) 화면이 기본 제공되지 않습니다.
- 대규모 프로젝트에서는 프로젝트 구조를 직접 설계해야 합니다.
- 비동기 프로그래밍(async/await)을 이해하지 못하면 활용이 제한될 수 있습니다.
- Python 생태계에 익숙하지 않으면 초기 학습이 필요합니다.

12. FastAPI 활용 사례

FastAPI 는 다음과 같은 분야에서 널리 활용됩니다.

- AI 및 LLM 서비스(API 서버)
- 챗봇 백엔드
- REST API 서버
- 모바일 앱 백엔드
- React, Vue, Angular 프론트엔드와 연동하는 백엔드
- IoT 데이터 수집 서버
- 실시간 데이터 처리 시스템
- 게시판, 회원관리, 쇼핑몰 등 웹 서비스
- 머신러닝 및 딥러닝 모델 추론(API 제공)

[FastAPI 로 자유게시글(Board) 서비스 백엔드 구현 실습]

FastAPI 는 Python 타입 힌트를 기반으로 API 를 만들고, /docs 에서 Swagger UI 문서를 자동 생성합니다. MySQL 같은 관계형 DB 는 SQLAlchemy 계열 ORM 으로 연동할 수 있습니다.

1. PyCharm 프로젝트 준비

1 단계: 프로젝트 생성

PyCharm 에서 다음 순서로 진행합니다.

File 메뉴

- New Project
- Location: fastapi_board_mysql_project
- Python 3.11 선택
- Create

2 단계: 가상환경 생성

PyCharm Terminal 에서 실행합니다.

```
python -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip setuptools wheel
```

3 단계: 패키지 설치

```
pip install -r requirements.txt
```

필요 패키지는 다음과 같습니다.

```
fastapi
uvicorn[standard]
sqlalchemy
pymysql
python-dotenv
passlib[bcrypt]
python-jose[cryptography]
pydantic
```

2. MySQL 준비

MySQL Workbench 또는 CMD 에서 DB 를 생성합니다.

```
CREATE DATABASE IF NOT EXISTS fastapi_board_db
DEFAULT CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;
```

.env 파일을 만들고 DB 접속 정보를 작성합니다.

```
DATABASE_URL=mysql+pymysql://root:1234@localhost:3306/fastapi_board_db?charset=utf8mb4
SECRET_KEY=fastapi-board-secret-key
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60
```

root:1234 부분은 본인 MySQL 계정과 비밀번호로 바꿉니다.

3. 프로젝트 구조

```
fastapi_board_mysql_project/
├── app/
│   ├── main.py
│   ├── database.py
│   ├── models.py
│   ├── schemas.py
│   ├── auth.py
│   └── routers/
│       ├── users.py
│       └── posts.py
├── .env
├── .env.example
├── .gitignore
├── requirements.txt
├── mysql_init.sql
└── README.md
```

역할은 다음과 같습니다.

파일	역할
main.py	FastAPI 앱 시작 파일
database.py	MySQL 연결 설정
models.py	DB 테이블 모델
schemas.py	요청/응답 데이터 구조
auth.py	비밀번호 암호화, JWT 로그인 처리
users.py	회원가입, 로그인 API
posts.py	게시글 CRUD API

4. 실행 방법

PyCharm Terminal 에서 실행합니다.

```
python -m uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

정상 실행되면 다음 주소로 접속합니다.

```
http://127.0.0.1:8000/docs
```

FastAPI 는 OpenAPI 기반 문서를 자동 생성하고, Swagger UI 는 기본적으로 /docs 에서 사용할 수 있습니다.

5. API 기능 구성

회원가입 : POST /users/signup

요청 예시:

```
{
  "username": "user01",
  "password": "1234",
  "name": "홍길동"
}
```

처리 흐름:

```
사용자 정보 입력
→ username 중복 확인
→ 비밀번호 bcrypt 해싱
→ users 테이블 저장
→ 회원가입 완료 응답
```

비밀번호는 원문 그대로 저장하지 않고 해시값으로 저장해야 합니다. FastAPI 공식 보안 튜토리얼도 JWT 와 비밀번호 해싱 구조를 안내합니다. ([FastAPI](#))

로그인: POST /users/login

요청 예시:

```
{
  "username": "user01",
  "password": "1234"
}
```

처리 흐름:

```
username 조회
→ 비밀번호 검증
→ 성공 시 JWT access_token 발급
→ 이후 게시글 등록/수정/삭제에서 토큰 사용
```

응답 예시:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs...\"",
  "token_type": "bearer"
}
```

게시글 등록 : POST /posts

인증 필요합니다.

요청 예시:

```
{
  "title": "첫 번째 게시글",
  "content": "FastAPI 와 MySQL 게시판 테스트입니다."
}
```

처리 흐름:

```
JWT 토큰 확인
→ 로그인 사용자 확인
→ posts 테이블에 제목/내용/작성자 저장
```

게시글 전체 조회 : GET /posts

처리 흐름:

posts 테이블 전체 조회

- 최신순 정렬
- 목록 반환

게시글 상세 조회 + 조회수 1 증가 : GET /posts/{post_id}

처리 흐름:

post_id 로 게시글 조회

- 없으면 404
- `view_count = view_count + 1`
- DB 반영
- 게시글 상세 반환

게시글 수정 : PUT /posts/{post_id}

인증 필요합니다.

요청 예시:

```
{  
  "title": "수정된 제목",  
  "content": "수정된 내용입니다."  
}
```

처리 흐름:

JWT 토큰 확인

- 게시글 존재 확인
- 작성자 본인인지 확인
- 제목/내용 수정

게시글 삭제 : DELETE /posts/{post_id}

인증 필요합니다.

처리 흐름:

```
JWT 토큰 확인
→ 게시글 존재 확인
→ 작성자 본인인지 확인
→ 게시글 삭제
```

6. Swagger 테스트 순서

1 단계: Swagger 접속

브라우저에서 접속합니다. : <http://127.0.0.1:8000/docs>

2 단계: 회원가입 테스트

Swagger 에서 다음 API 를 엽니다. : POST /users/signup

Try it out 클릭 후 입력합니다.

```
{
  "username": "user01",
  "password": "1234",
  "name": "홍길동"
}
```

Execute 를 누릅니다.

3 단계: 로그인 테스트

POST /users/login

입력합니다.

```
{
  "username": "user01",
  "password": "1234"
}
```

응답에서 access_token 값을 복사합니다.

4 단계: Swagger 인증 설정

Swagger 상단의 Authorize 버튼을 클릭합니다.

입력창에 다음처럼 넣습니다.

```
Bearer 복사한_access_token
```

예:

```
Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6I...
```

Authorize → Close 를 누릅니다.

5 단계: 게시글 등록

POST /posts

요청 Body:

```
{
  "title": "첫 번째 게시글",
  "content": "FastAPI 게시판 등록 테스트입니다."
}
```

Execute 를 누릅니다.

6 단계: 게시글 전체 조회

GET /posts

Execute 를 누르면 전체 게시글 목록이 출력됩니다.

7 단계: 게시글 상세 조회

GET /posts/{post_id}

post_id 에 1 을 넣고 실행합니다.

실행할 때마다 view_count 가 1 씩 증가합니다.

8 단계: 게시글 수정

PUT /posts/{post_id}

post_id 에 1 을 넣고 Body 를 작성합니다.

```
{
  "title": "수정된 게시글 제목",
  "content": "수정된 게시글 내용입니다."
}
```

로그인한 사용자가 본인이 작성한 글만 수정할 수 있습니다.

9 단계: 게시물 삭제

`DELETE /posts/{post_id}`

`post_id` 에 1 을 넣고 실행합니다.

로그인한 사용자가 작성한 글만 삭제할 수 있습니다.

7. 실행 확인 순서 요약

1. MySQL 실행
2. `fastapi_board_db` 생성
3. PyCharm 에서 프로젝트 열기
4. `.env` 작성
5. 가상환경 활성화
6. `pip install -r requirements.txt`
7. `uvicorn` 실행
8. `/docs` 접속
9. 회원가입
10. 로그인
11. Authorize 에 Bearer 토큰 입력
12. 게시물 등록
13. 전체 조회
14. 상세 조회
15. 수정
16. 삭제